



Project Architecture & Development Blueprint

Flutter Mobile Application + ASP.NET Core Web API + PostgreSQL

Project Detail	Value
Academy	Shri Nrityalaya – School of Bharatanatyam Dance
Frontend	Flutter – Teacher, Student and Parent mobile experiences
Backend	ASP.NET Core Web API
Database	PostgreSQL
Core Design	Single-teacher-first, simple daily operation, scalable architecture

DESIGN PRINCIPLE

The academy is currently operated by one teacher who teaches and manages the complete business. The system therefore hides technical complexity behind fast daily workflows, bulk actions and a practical 'Action Required' dashboard.

Document Contents

1. 1. Project Vision & Objectives
2. 2. Business Model & User Roles
3. 3. Complete System Architecture
4. 4. Brand Theme & UI Design System
5. 5. Functional Modules
6. 6. Detailed User Flows
7. 7. Flutter Frontend Architecture
8. 8. Complete Flutter File Structure
9. 9. .NET Backend Architecture
10. 10. Complete Backend File Structure
11. 11. Database Architecture
12. 12. REST API Design
13. 13. Authentication & Authorization
14. 14. Fees & Manual Payment Workflow
15. 15. File Storage & Media Strategy
16. 16. Notifications & Communication
17. 17. Reports & Analytics
18. 18. Arangetram & Events
19. 19. Security & Audit
20. 20. Development Phases
21. 21. Testing Strategy
22. 22. Deployment
23. 23. Configuration & Secrets
24. 24. Future Scalability
25. 25. Final Build Checklist

1. Project Vision & Objectives

Shri Nrityalaya Academy Management System is a full-stack mobile application for managing a Bharatanatyam academy. It centralizes student records, batches, schedules, attendance, fees, payment verification, lessons, practice, progress, events, performances and Arangetram preparation.

Objective	Implementation
Reduce manual work	Quick actions, bulk attendance, reminders and pending-action dashboard.
Centralize academy records	Single source of truth for students, parents, fees and learning history.
Support parents	Attendance, progress, fees, announcements and payment proof submission.
Support learning	Lessons, recordings, practice assignments, video submissions and feedback.
No payment gateway dependency	Manual UPI/bank proof upload plus teacher approval and direct physical payment entry.
Future-ready	Supports additional teachers, branches and payment providers later.

Success Criteria

- Teacher can take attendance for a batch in under one minute.
- Teacher immediately sees payment proofs and other actions requiring attention.
- Parents can submit transaction proof without an integrated payment gateway.
- Teacher can record cash or physical payment directly.
- Student progress is historical and visible.
- The application remains simple enough for one person to operate every day.

2. Business Model & User Roles

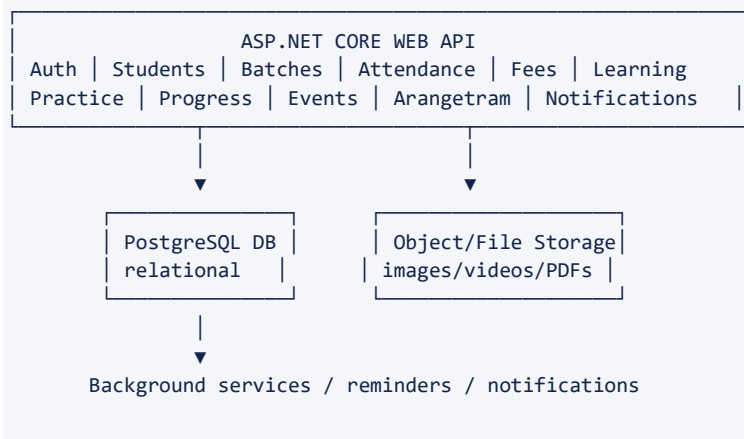
CURRENT OPERATING MODEL

One academy • one primary teacher/owner • multiple batches • students and parents. Do not introduce unnecessary accountant, attendance-manager or event-manager workflows in Version 1.

Role	Purpose	Permissions
System Admin	Technical platform management	Academy provisioning, support, technical configuration, system audit.
Teacher / Academy Owner	Primary operational user	Full control of students, batches, attendance, fees, lessons, progress, events, Arangetram, certificates and announcements.
Student	Learning user	View schedule and lessons, submit practice, view attendance/progress/certificates.
Parent	Guardian user	View linked student, fee status, payment history, submit payment proof, view announcements and progress.

3. Complete System Architecture





Layer	Technology	Reason
Mobile	Flutter + Dart	Single Android/iOS codebase.
State	Riverpod	Scalable and testable state management.
Routing	go_router	Role-aware routing and clean navigation.
HTTP	Dio	Interceptors, token refresh and robust networking.
Backend	ASP.NET Core Web API	Strong authentication, validation and maintainable APIs.
ORM	Entity Framework Core	Migrations and maintainable database access.
Database	PostgreSQL	Reliable relational storage and flexible hosting.
Authentication	JWT + Refresh Tokens	Secure API sessions.
File Storage	S3-compatible object storage	Keep large files outside database.
Notifications	Firebase Cloud Messaging	Push notifications.
Logging	Serilog	Structured server logging.
API Docs	Swagger / OpenAPI	Frontend integration and testing.

4. Brand Theme & UI Design System

The uploaded Shri Nrityalaya logo establishes the visual identity: deep navy, luminous gold and traditional Bharatanatyam imagery. The app should feel premium, elegant and cultural—not like a generic school ERP.

Token	Value	Usage
Primary Navy	#142448	Main backgrounds, app bars and strong surfaces.
Deep Navy	#0E1A35	Dark gradients and splash/login background.
Primary Gold	#F5C542	Primary actions, selected states and premium accents.
Light Gold	#FFD968	Gradient highlight and decorative accents.
Warm Cream	#FFF8E8	Light cards and premium content surfaces.
Text Primary	#26334A	Readable body text.
Muted	#667085	Secondary text.
Success	#2E8B57	Paid/approved/completed.
Warning	#D97706	Pending/under review.
Error	#C2413B	Rejected/overdue/destructive.

UI Direction

- Use navy → deep navy gradients for splash, authentication and important hero sections.
- Use gold as a deliberate accent; avoid making every component yellow.
- Use subtle lotus, temple, mudra or Bharatanatyam line-art at low opacity.
- Use rounded cards, generous spacing and simple typography.
- Keep the teacher dashboard focused on today's work and pending actions.

- Build reusable status chips: Paid, Pending, Under Review, Partial, Rejected, Present, Absent, Learning and Mastered.

```
lib/core/theme/
├── app_colors.dart
├── app_theme.dart
├── app_typography.dart
├── app_spacing.dart
└── app_component_theme.dart
```

5. Functional Modules

Module	Responsibilities
Dashboard	Today's classes, pending payment reviews, pending fees, practice reviews, upcoming events and quick actions.
Students	Profile, joining status, batch, parent links, notes and history.
Parents	Contact details and linked student access.
Batches	Level, schedule, students, fee defaults and status.
Class Sessions	Actual class date/time, conducted/cancelled/rescheduled status.
Attendance	Present/absent/late/leave and reports.
Fees	Fee plans, monthly obligations, partial payments and balances.
Payments	UPI/bank proof upload, transaction reference, teacher approval/rejection and physical payment.
Learning	Lessons, recordings, resources and batch access.
Practice	Assignments, optional video submission and teacher feedback.
Progress	Adavu, Margam and teacher notes.
Arangetram	Candidates, repertoire, rehearsals and readiness checklist.
Events	Participants, rehearsals, requirements, media and certificates.
Announcements	Targeted communication by batch/student/parent.
Notifications	In-app and push notifications.
Certificates	Event/course/Arangetram certificates as PDFs.
Reports	Student, attendance, fees, progress and operational reports.

6. Detailed User Flows

Teacher Daily Flow

Login → Teacher Dashboard → Today's Classes → Open Batch → Mark Attendance → Save → Review pending payments/practice → Complete other actions.

Student Creation

Teacher → Add Student → Student Details → Parent Details → Assign Batch → Assign Fee Plan → Save → Account access activated.

Parent Payment Proof

Parent → Fees → Select Month → Enter Amount/Method/Reference → Upload Screenshot → Submit → Under Review → Teacher Approves/Rejects → Parent Notified.

Physical Payment

Teacher → Student → Fees → Record Payment → Cash/Physical/Bank → Amount/Date → Save → Paid or Partial.

Attendance

Teacher → Today's Class → Mark All Present → Change exceptions → Save → History updated → Optional notification.

Practice

Teacher creates assignment → Student practices → optional video upload → Teacher reviews → feedback → progress updated.

Event

Create Event → Select Participants → Add Requirements → Rehearsals → Track Readiness → Performance → Media/Certificates.

7. Flutter Frontend Architecture

Recommended approach: Feature-First Clean Architecture with Riverpod.

```

Presentation
  Screens / Widgets / Controllers
    |
Application
  Use Cases / Providers / State
    |
Domain
  Entities / Repository Contracts
    |
Data
  Models / Data Sources / Repository Implementations
    |
Infrastructure
  Dio / Secure Storage / Firebase / File Services
  
```

- Widgets must not call HTTP APIs directly.
- API access belongs to data sources/services.
- Repositories isolate API implementation.
- Riverpod controllers manage screen state.
- Use centralized error mapping and user-friendly messages.
- Frontend role checks are for UX only; backend authorization is the real security boundary.

8. Complete Flutter File Structure

```

shri_nrityalaya_app/
├── android/
├── ios/
├── assets/
│   ├── images/
│   │   ├── academy_logo.png
│   │   ├── splash_background.png
│   │   └── placeholders/
  
```

```

├── icons/
├── fonts/
├── lib/
│   ├── main.dart
│   ├── bootstrap.dart
│   ├── app/
│   │   ├── app.dart
│   │   ├── app_router.dart
│   │   └── app_providers.dart
│   ├── core/
│   │   ├── config/
│   │   │   ├── app_config.dart
│   │   │   └── environment.dart
│   │   ├── constants/
│   │   │   ├── api_constants.dart
│   │   │   ├── app_constants.dart
│   │   │   └── storage_keys.dart
│   │   ├── errors/
│   │   │   ├── app_exception.dart
│   │   │   ├── failure.dart
│   │   │   └── error_mapper.dart
│   │   ├── network/
│   │   │   ├── dio_client.dart
│   │   │   ├── auth_interceptor.dart
│   │   │   └── refresh_token_interceptor.dart
│   │   ├── storage/secure_storage_service.dart
│   │   ├── theme/
│   │   │   ├── app_colors.dart
│   │   │   ├── app_theme.dart
│   │   │   ├── app_typography.dart
│   │   │   └── app_spacing.dart
│   │   ├── utils/
│   │   │   ├── date_utils.dart
│   │   │   ├── currency_utils.dart
│   │   │   └── validators.dart
│   │   └── widgets/
│   │       ├── app_loader.dart
│   │       ├── app_empty_state.dart
│   │       ├── app_error_view.dart
│   │       ├── app_confirm_dialog.dart
│   │       └── status_badge.dart
│   ├── features/
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── students/
│   │   ├── parents/
│   │   ├── batches/
│   │   ├── schedule/
│   │   ├── attendance/
│   │   ├── fees/
│   │   ├── payments/
│   │   ├── lessons/
│   │   ├── practice/
│   │   ├── progress/
│   │   ├── arangetram/
│   │   ├── events/
│   │   ├── announcements/
│   │   ├── notifications/
│   │   ├── certificates/
│   │   ├── reports/
│   │   └── profile/
│   └── shared/
│       ├── models/
│       ├── widgets/
│       └── extensions/

```

```

├── test/
├── integration_test/
├── pubspec.yaml
├── analysis_options.yaml
└── README.md

```

Each feature follows:

```

feature/
├── data/
│   ├── datasources/
│   ├── models/
│   └── repositories/
├── domain/
│   ├── entities/
│   ├── repositories/
│   └── usecases/
└── presentation/
    ├── controllers/
    ├── screens/
    └── widgets/

```

Purpose	Recommended Package
State	flutter_riverpod / riverpod_annotation
Routing	go_router
HTTP	dio
Secure storage	flutter_secure_storage
Models	freezed + json_serializable
Files	file_picker + image_picker
Image cache	cached_network_image
Push	firebase_core + firebase_messaging
Local notifications	flutter_local_notifications
Video	video_player
Connectivity	connectivity_plus
Formatting	intl
Permissions	permission_handler

9. .NET Backend Architecture

Recommended approach: Clean Architecture / layered architecture.

```

API Layer
Controllers + Authentication + HTTP contracts
|
Application Layer
Use Cases + Services + DTOs + Validation
|
Domain Layer
Entities + Enums + Business Contracts
|
Infrastructure Layer
EF Core + PostgreSQL + Storage + FCM + Background Jobs

```

- Controllers remain thin.
- Business rules belong in application services/use cases.
- Do not expose EF entities directly; use request/response DTOs.
- Use FluentValidation for write requests.
- Use global exception middleware.
- Use EF Core migrations for all schema changes.

- Store timestamps in UTC.

10. Complete Backend File Structure

```

ShriNrityalaya.sln
├── src/
│   ├── ShriNrityalaya.Api/
│   │   ├── Controllers/
│   │   │   ├── AuthController.cs
│   │   │   ├── DashboardController.cs
│   │   │   ├── StudentsController.cs
│   │   │   ├── ParentsController.cs
│   │   │   ├── BatchesController.cs
│   │   │   ├── ClassSessionsController.cs
│   │   │   ├── AttendanceController.cs
│   │   │   ├── FeePlansController.cs
│   │   │   ├── PaymentsController.cs
│   │   │   ├── LessonsController.cs
│   │   │   ├── PracticeController.cs
│   │   │   ├── ProgressController.cs
│   │   │   ├── ArangetramController.cs
│   │   │   ├── EventsController.cs
│   │   │   ├── AnnouncementsController.cs
│   │   │   ├── NotificationsController.cs
│   │   │   ├── CertificatesController.cs
│   │   │   └── ReportsController.cs
│   │   ├── Middleware/
│   │   │   ├── ExceptionHandlingMiddleware.cs
│   │   │   └── RequestLoggingMiddleware.cs
│   │   ├── Extensions/
│   │   ├── Program.cs
│   │   └── appsettings.json
│   ├── ShriNrityalaya.Application/
│   │   ├── Common/
│   │   ├── Authentication/
│   │   ├── Dashboard/
│   │   ├── Students/
│   │   ├── Parents/
│   │   ├── Batches/
│   │   ├── Attendance/
│   │   ├── Fees/
│   │   ├── Payments/
│   │   ├── Learning/
│   │   ├── Practice/
│   │   ├── Progress/
│   │   ├── Arangetram/
│   │   ├── Events/
│   │   ├── Communication/
│   │   ├── Certificates/
│   │   └── Reports/
│   ├── ShriNrityalaya.Domain/
│   │   ├── Common/
│   │   ├── Entities/
│   │   ├── Enums/
│   │   ├── ValueObjects/
│   │   └── Interfaces/
│   └── ShriNrityalaya.Infrastructure/
│       ├── Persistence/
│       │   ├── ApplicationDbContext.cs
│       │   ├── Configurations/
│       │   ├── Migrations/
│       │   └── Seed/
│       ├── Identity/
│       └── Authentication/

```

```

├── Storage/
├── Notifications/
├── BackgroundJobs/
├── Logging/
├── tests/
│   ├── ShriNrityalaya.Application.Tests/
│   ├── ShriNrityalaya.Api.Tests/
│   └── ShriNrityalaya.IntegrationTests/
├── docker/
├── docs/
├── docker-compose.yml
└── README.md

```

11. Database Architecture

Core rule: PostgreSQL stores relational data; object storage stores large files.

Area	Tables
Identity	Users, Roles, UserRoles, RefreshTokens, DeviceTokens
Academy	Academies, Teachers
People	Students, Parents, StudentParents
Batches	Batches, BatchStudents, ClassSchedules, ClassSessions
Attendance	AttendanceDetails
Fees	FeePlans, StudentFeePlans, FeeObligations, Payments, PaymentProofs, Receipts
Learning	Lessons, LessonResources, ClassRecordings
Practice	PracticeAssignments, PracticeSubmissions, PracticeFeedback
Progress	Adavus, StudentAdavuProgress, MargamItems, StudentMargamProgress, ProgressNotes
Arangetram	ArangetramCandidates, ArangetramRepertoire, Rehearsals, ArangetramChecklists
Events	Events, EventParticipants, EventRehearsals, EventRequirements, EventMedia
Communication	Announcements, AnnouncementRecipients, Notifications
Certificates	Certificates, CertificateTemplates
System	AuditLogs, FileMetadata

```

Academy
├── Teacher
│   ├── Batch —< BatchStudents >— Student —< StudentParents >— Parent
│   ├── ClassSession —< AttendanceDetail >— Student
│   ├── FeePlan —< StudentFeePlan >— Student
│   ├── FeeObligation —< Payment >— Student
│   ├── Lesson — assigned to — Batch
│   ├── PracticeAssignment —< PracticeSubmission >— Student
│   ├── StudentAdavuProgress
│   ├── StudentMargamProgress
│   ├── ArangetramCandidate
│   └── Event —< EventParticipant >— Student

```

Recommended Payment Model

```

FeePlan
  Id, Name, Amount, Frequency, Active

StudentFeePlan
  StudentId, FeePlanId, EffectiveFrom

FeeObligation

```

StudentId, BillingMonth, ExpectedAmount, DueDate, Status

Payment

FeeObligationId, Amount, Method, PaidAt,
TransactionReference, Status, RecordedByUserId

PaymentProof

PaymentId, FileMetadataId

Receipt

PaymentId, ReceiptNumber, GeneratedAt

12. REST API Design

Use a versioned base path such as /api/v1 and consistent response/error models.

Resource	Representative Endpoints
Auth	POST /auth/login • /auth/request-otp • /auth/verify-otp • /auth/refresh • /auth/logout
Dashboard	GET /dashboard/teacher • /dashboard/student • /dashboard/parent
Students	GET/POST /students • GET/PUT/DELETE /students/{id}
Parents	GET/POST /parents • GET/PUT /parents/{id}
Batches	GET/POST /batches • PUT /batches/{id} • POST /batches/{id}/students
Class Sessions	GET/POST /class-sessions • PUT /class-sessions/{id}/cancel
Attendance	GET /attendance/session/{id} • PUT /attendance/session/{id}
Fees	GET/POST /fee-plans • POST /students/{id}/fee-plan • GET /fees/obligations
Payments	POST /payments • POST /payments/{id}/proof • /approve • /reject
Lessons	GET/POST /lessons • POST /lessons/{id}/resources
Practice	GET/POST /practice-assignments • POST /practice-submissions/{id}/feedback
Progress	GET/PUT student Adavu and Margam progress
Arangetram	Candidate, repertoire, rehearsal and checklist endpoints
Events	GET/POST /events • participants • rehearsals
Announcements	GET/POST /announcements
Certificates	POST /certificates/generate • GET /certificates/{id}/download
Reports	GET /reports/fees • /attendance • /students • /progress

Success:

```
{ "success": true, "message": "Operation completed.", "data": {} }
```

Error:

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": { "field": ["Message"] },
  "traceId": "..."
}
```

13. Authentication & Authorization

Mobile / Email

↓

Password or OTP

↓

Identity Validation

↓

Short-lived JWT Access Token

+

```

Refresh Token
  ↓
Role Claims
Teacher / Student / Parent
  ↓
Authorized API Access

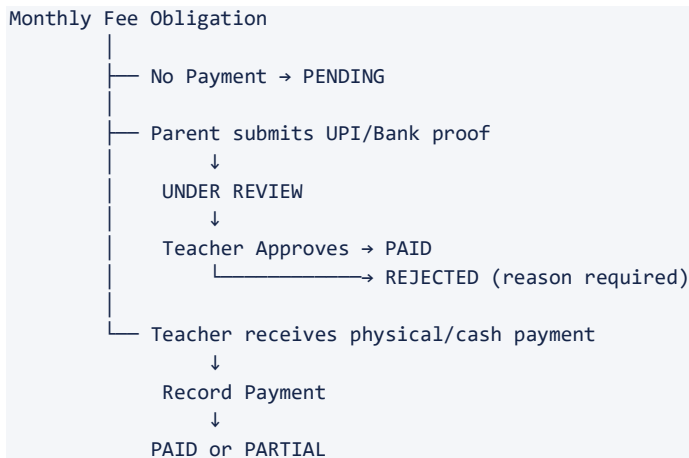
```

- Store tokens with flutter_secure_storage.
- Use short-lived JWT access tokens and server-side refresh token tracking.
- OTP must expire and be rate-limited.
- Passwords use secure ASP.NET Core Identity hashing.
- Backend must validate role plus academy/resource ownership.
- Frontend role-based UI never replaces backend authorization.

14. Fees & Manual Payment Workflow

VERSION 1 PAYMENT POLICY

The application records and verifies payments; it does not process money. Parents pay through UPI/bank outside the app and submit proof. Teacher can record cash or physical payment.



Status	Meaning
PENDING	Expected fee exists but no verified payment.
UNDER REVIEW	Parent submitted proof awaiting teacher action.
PAID	Full required amount received and verified/recorded.
PARTIAL	Some amount received; remaining balance exists.
REJECTED	Proof rejected; rejection reason stored.
WAIVED	Teacher intentionally removes the obligation where allowed.

15. File Storage & Media Strategy

Never store large videos or image binaries directly inside PostgreSQL.

```

Flutter Upload
  ↓
ASP.NET API validates user + file
  ↓
Object Storage

```



FileMetadata record in PostgreSQL

FileMetadata:

Id, OwnerUserId, StorageKey, OriginalFileName,
ContentType, FileSize, Category, CreatedAt

Type	Examples	Handling
Images	Payment proof, student profile	Validate MIME type/size; compress where suitable.
Videos	Class recording, practice video	Object storage with limits; CDN/transcoding can be added later.
PDF	Certificates	Generate server-side and authorize download.
Documents	Learning material	Allow approved formats only.

16. Notifications & Communication

Trigger	Recipient	Message
Payment submitted	Teacher	New payment requires review.
Payment approved	Parent	Payment approved.
Payment rejected	Parent	Payment rejected with reason.
Fee reminder	Parent	Configurable pending fee reminder.
Announcement	Selected audience	New academy announcement.
Class changed	Batch/parents/students	Cancellation or schedule change.
Practice feedback	Student/parent	Teacher reviewed practice submission.
Event reminder	Participants	Upcoming rehearsal/performance.

Version 1: In-app notifications + Firebase Cloud Messaging. WhatsApp/email integration should remain a future enhancement.

17. Reports & Analytics

Report	Information
Teacher Dashboard	Today's classes, pending payments, practice reviews and upcoming events.
Student Report	Attendance, fee status, batch and progress.
Attendance Report	Percentage by student, batch and date range.
Fee Report	Expected, collected, pending, partial and under-review.
Progress Report	Adavu/Margam completion and notes.
Event Report	Participants, rehearsals and readiness.
Arangetram Report	Candidate readiness and checklist completion.

KEEP ANALYTICS PRACTICAL

For a single-teacher academy, actionable information is more useful than complex enterprise charts.

18. Arangetram & Event Management

Teacher marks Student as Arangetram Candidate

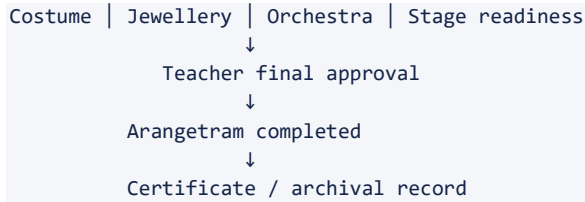


Candidate preparation profile



Repertoire | Rehearsals | Checklist





Event Flow

Create Event → Select Participants → Add Requirements →
 Schedule Rehearsals → Track Readiness → Performance →
 Upload Media (optional) → Generate Certificates (optional)

19. Security, Validation & Audit

- HTTPS for production.
- JWT + refresh token revocation.
- Role and resource-level authorization.
- Rate limiting for login/OTP.
- FluentValidation for write requests.
- File type and size validation.
- Never expose object-storage credentials to Flutter.
- Use signed URLs or authorized download endpoints for protected media.
- Structured logging without passwords, OTPs or tokens.
- Audit payment approval/rejection, fee edits, attendance edits and certificate generation.
- Use soft-delete/status archival for records requiring historical preservation.

20. Development Phases & Milestones

Phase	Scope	Outcome
Phase 0	Requirements, UX, DB and API contract	Approved foundation before coding.
Phase 1 – Core	Auth, roles, dashboard, students, parents, batches, schedules	Academy data management works.
Phase 2 – Daily Operations	Class sessions, attendance, fee plans, manual payments, payment proof review	Teacher can run daily operations.
Phase 3 – Learning	Lessons, recordings, practice and feedback	Digital learning workflow.
Phase 4 – Progress	Adavu/Margam progress and reports	Student development tracking.
Phase 5 – Events	Events, rehearsals, requirements and certificates	Performance management.
Phase 6 – Arangetram	Candidate tracking and readiness	Advanced academy workflow.
Phase 7	Notifications, security hardening, testing and deployment	Production release.

21. Testing Strategy

Level	Coverage
Flutter Unit	Validators, controllers and state.
Flutter Widget	Forms, screens and error states.
Flutter Integration	Login, payment submission and end-to-end journeys.
.NET Unit	Business rules and calculations.
.NET Integration	API, database and authorization workflows.
Manual UAT	Real teacher and parent workflows.

- Parent cannot access another student's information.
- Payment cannot be approved twice.
- Partial payment balance is correct.
- Rejected payment does not count as collected revenue.
- Attendance belongs to the correct session and student.
- Expired refresh token requires authentication.
- Invalid uploads are rejected.
- Notification failure does not break the main transaction.

22. Deployment & Environments

Environment	Purpose
Development	Local development and test data.
Staging	Pre-production testing.
Production	Live API, database, storage and notifications.

```

Development
Flutter → Local ASP.NET API → Local PostgreSQL

Production
Flutter Release App → HTTPS ASP.NET API
                        ├── PostgreSQL
                        ├── Object Storage
                        └── Firebase Cloud Messaging

```

- Containerize the ASP.NET Core API with Docker.
- Use environment variables/secrets in production.
- Configure automated PostgreSQL backups.
- Add health checks and structured logs.
- Restrict CORS to required clients/origins.
- Add CI/CD after the system is stable.

23. Configuration & Secrets

```

appsettings.json
{
  "ConnectionStrings": { "DefaultConnection": "" },
  "Jwt": { "Issuer": "", "Audience": "", "Secret": "" },
  "Storage": { "Provider": "", "Bucket": "" },
  "Firebase": { "ProjectId": "" }
}

```

- Never commit production passwords or JWT secrets.
- Use .NET User Secrets locally where appropriate.
- Use deployment secret management/environment variables.
- Separate development and production configurations.

24. Future Scalability

Future Need	Prepared Architecture
-------------	-----------------------

More Teachers	Add academy membership and permissions.
Multiple Branches	Add Branch and branch-aware ownership.
Payment Gateway	Add provider integration while retaining current Payment model.
WhatsApp	Implement communication provider abstraction.
Web Admin Portal	Reuse the same ASP.NET Core API.
Advanced Video	Add transcoding/CDN without changing relational metadata.
Multiple Academies	Use AcademyId to isolate tenant data.

25. Final Build Checklist

Foundation

- ☐ Project repositories
- ☐ Flutter setup
- ☐ ASP.NET solution
- ☐ PostgreSQL
- ☐ API contract

Branding

- ☐ Logo
- ☐ Navy/gold theme
- ☐ Typography
- ☐ Reusable components

Identity

- ☐ Teacher login
- ☐ Parent/student access
- ☐ JWT refresh
- ☐ Role routing
- ☐ Backend authorization

Core

- ☐ Students
- ☐ Parents
- ☐ Batches
- ☐ Schedules
- ☐ Dashboard

Operations

- ☐ Class sessions
- ☐ Attendance
- ☐ Fee plans
- ☐ Monthly obligations
- ☐ Manual payment
- ☐ Proof approval
- ☐ Partial payments

Learning

- ☐ Lessons
- ☐ Recordings
- ☐ Practice
- ☐ Feedback

Progress

- ☐ Adavu
- ☐ Margam
- ☐ Progress notes

Advanced

- ☐ Events
- ☐ Certificates
- ☐ Arangetram

Production

- ☐ Notifications
- ☐ Audit logs
- ☐ Tests
- ☐ Backups
- ☐ HTTPS
- ☐ Monitoring

Final Architecture Recommendation

RECOMMENDED FINAL BUILD

Flutter + Riverpod + go_router + Dio for the mobile app; ASP.NET Core Web API + Entity Framework Core for the backend; PostgreSQL for relational data; S3-compatible object storage for payment screenshots, recordings and practice videos; Firebase Cloud Messaging for push notifications.

“Enterprise-quality architecture behind a graceful, simple and traditional Bharatanatyam experience.”